

AI-POWERED FORM FILLING ASSISTANT FOR INDIAN CITIZEN SERVICES

Submitted as a partial fulfilment of Bachelor of Technology in Computer Science &
Engineering

of

Maulana Abul Kalam Azad University of Technology
(Formerly known as West Bengal University of Technology)



Summer Internship Report *Submitted by*

Name of Student(s)	University Roll No.
Satakshi Poddar	11600222096
Priyal Banerjee	11600222073
Tanya Shaw	11600223011

Conducted at

INTEL®



**Department of Computer Science & Engineering,
MCKV Institute of Engineering
243, G.T. Road(N)**

Liluah, Howrah - 711204

ACKNOWLEDGEMENT

The authors thank the mentors and coordinators of Intel Unnati program for their continuous support, constructive feedback and technical guidance throughout this work. The authors also thank the faculty and administration of their institution, in particular their project mentor and Assistant Professor, Dr. Deep Suman Dev, for the encouragement and for providing the necessary infrastructure to conduct this internship project. Finally, the authors acknowledge the efforts of all team members, whose collaboration and commitment contributed to the successful completion of this project.

The authors also acknowledge all the volunteers who enthusiastically helped in the dataset creation process by providing their personal documents and that all the data used in the prescribed datasets were collected and replicated from them with prior informed consent. The dataset does not intentionally include data from individuals outside this group, and any resemblance to external individuals or entities is purely coincidental.

CONTENTS

S. No	Content	Page No.
1	Abstract	1
2	Introduction	2-4
3	Training Works undertaken	5
4	Related Work	6-8
5	Datasets	9
6	Development Environment	10
7	Methodology	11-16
8	System Highlights	17-19
9	Results and Discussions	20-27
10	Special Terminologies used	28
11	Conclusion	29-30
12	Future scope of the work	31
13	References	32-36
14	Appendix	37-38

ABSTRACT

Accomplishing government paperwork in India frequently turns out to be a complex process for many citizens due to the extensive and complex format of government forms, technical terminologies, overwhelming procedures, language barriers, physical accessibility challenges and low digital literacy particularly in rural areas complicating the task further. Digital automation using AI models practically simplifies this process, but current systems focus primarily on image-based, layout-based, agentic or Blackbox approaches, making them brittle, difficult to enhance and complex for edge deployments. Data scarcity is the most overwhelming factor that hampers the performance of current systems and their English centric design restricts their wide scale adaption in India. This project proposes the Governmental Form-Submissions via AI (GovAI-Sub) web application, integrating the Indic-Vision-ASR-NER AI (IVAN AI), an end-to-end novel solution to these problems in the form of a robust multilingual, multimodal, modular, light weight and easy to enhance system, employing Optical Character Recognition (OCR), Automatic Speech Recognition (ASR), Machine Translation and Named Entity Recognition (NER) to analyze and extract information from document images or speech and automatically populate the forms. GovAI-Sub and IVAN AI addresses critical gaps in existing research by providing an efficient and integrated multilingual solution specifically designed for Indian government forms with minimal data requirements. The system achieves 77.81% accuracy on hybrid Paddle-Surya OCR, 86.55% on customized Machine Translation using PolyGlott and Facebook nllb-200-distilled-600M, 92.33% on ASR with OpenAI Whisper Large-v3, and 79.8% on NER with BERT-Large. GovAI-Sub with the integration of IVAN AI reduces human errors, language barriers and manual effort, delivering a user-friendly and inclusive digital solution for improved citizen service accessibility and adaptability across India.

INTRODUCTION

India with a population exceeding 1.45 billion [1], [2], faces significant challenges in delivering accessible government services despite digital advancements under the 2015 Digital India Initiative. While rural internet accessibility has reached 80.4% [2]–[4], digital literacy and a 74.04% general literacy rate marked by gender disparities remain critical concerns. This is compounded by immense linguistic diversity, 22 official languages are spoken by 96.71% of the population, with over 121 languages exceeding 10,000 speakers each [2]. Consequently, predominant manual form-filling for essential documents like Aadhaar, Voter, PAN, etc., remains a formidable barrier for citizens with low digital proficiency, disabilities or language constraints [2], [3]. Despite MeitY's [3], UX4G program workshops [5] and milestones like 43 crore DigiLocker users and 12,309 integrated services [3], [4], a gap persists in intelligent multilingual, multimodal automated form-filling systems, capable of processing voice, text and documents in Indian languages to ensure inclusive digital governance [2]–[4].

Problem Statement: These systemic barriers render government form-filling an arduous and error-prone process for average Indian citizens. The complexity is driven by lengthy structures, technical jargon, complex procedures, digital illiteracy and lack of clear instructions which frequently result in application rejections. While India is linguistically diverse, the reliance on English or Hindi-centric documentation necessitates that regional language speakers seek third-party assistance at cyber cafes or service centers incurring unnecessary costs and delays. Furthermore, manual data entry is plagued by illegible handwriting and incomplete submissions, leading to administrative bottlenecks and the potential denial of essential benefits. The absence of robust accessibility features and multilingual interfaces ultimately marginalizes aged and rural population or persons with disabilities, creating a significant disconnect between the current digital governance infrastructure and the goal of equitable service delivery.

Implemented Solution: To address these technical gaps, this project proposes the Indic-Vision-ASR-NER AI (IVAN AI), a modular, lightweight, easy to enhance, end-to-end multilingual and multimodal pipeline, along with the user friendly and easy-to-use Governmental Form-Submissions via AI (GovAI-Sub) web application Interface, designed for Indian government form automation. IVAN AI employs a dual-input architecture that supports document images or PDFs and voice-based input, ensuring accessibility for the elderly and users with motor, visual, vocal or auditory impairments. The document processing pipeline employs the IVAN-OCR, that utilizes a hybrid model combining the Paddle OCR [31] text detection module with the Surya OCR [36] text recognition module

for robust Indic text extraction supporting 17 and 21 Indian languages respectively. These extracted code-mixed texts are processed by the IVAN-Translator, a customized integration of PolyGlot [44] and light-weight nllb-200-distilled-600M [38] supporting 11 and 22 Indian languages respectively to translate information into single high-resource English language from auto detected source language of the information. This strategy allows the use of IVAN-NER (BERT-Doc-NER-Max), a domain-adapted BERT-Large-NER model [59] fine-tuned on a custom English-centric text dataset of 21 government-specific entities including Aadhaar, PAN, pension details, etc., and is easily expandable to more types. By shifting from complex multilingual image-based extraction to an English-centric text-space post-OCR, the system reduces manual annotation overhead on images and improves generalization. For voice based interactions, the system employs the IVAN-ASR leveraging the OpenAI Whisper Large-v3 [42] to transcribe raw user speech, supporting 12 major Indic languages, followed by the IVAN-Translator for translating the ASR-transcribed text to a single target language. This real-time ASR-Translator pipeline auto-interprets spoken information, transcribing and translating relevant field information directly for form-filling, bypassing the need for explicit NER extraction. The GovAI-Sub web-app designed to integrate this IVAN AI pipeline is made with easy-accessibility and simplicity in mind, to cater to a large group of population across language, age, physical and other barriers. The interface is made simple and easy-to-use providing a clear set of instructions in user desired language in a digital platform, that is easily accessible from anywhere. Validated across 9 government document types and 3 synthetic forms, mimicking the structural complexity of actual government forms to ensure legal compliance, IVAN AI achieves 77.8% accuracy on OCR, 86.5% on Translation, 92.3% on ASR, and 79.8% on NER. The overall system of GovAI-Sub and IVAN AI remains lightweight and modular for edge deployment, effectively addressing the scalability and data scarcity challenges prevalent in current literature, while maintaining the multimodality, simplicity of form-filling process, lingual and accessibility concerns of existing manual and digital governance frameworks.

The primary novelties of this work are summarized as follows:

- **The IVAN AI System:**

- Supports diverse Indic languages alongside English, addressing English-centric and Indic script limitations of existing systems.
- Integrates OCR-based document and ASR-based speech processing within a unified multimodal architecture.
- Employs modular architecture enabling seamless enhancements and debugging within an end-to-end pipeline, mitigating the limitations of Blackbox systems, VLMs and static systems.
- Lightweight, resource and cost-efficient models suitable for future edge deployment and offline use, countering limitations of agentic APIs and cloud services.

- Generalizes across 9 document types, validated on 3 synthetic forms with possible extensions, addressing domain-specific limitations of existing systems.
- Provides comprehensive benchmarks through real-world test cases for individual component-level evaluation.
- **Dataset Requirements:**
 - No dataset requirements for OCR, ASR and translation, leveraging transfer learning approach, thus eliminating fine-tuning needs.
 - Performs domain shift and translation from image to English-centric text space post-OCR, exploiting text processing simplicity over image manipulation and NER model fine-tuning on easily replicable English-text datasets, mitigating the requirements and labor-intensive annotations of image-based or Indic datasets.
- **IVAN-OCR Component:**
 - Robust hybrid algorithm combining text detection module of Paddle OCR with the text recognition module of Surya OCR, capable of code-mixed Indic text extraction from poor quality, degraded, complex document images having skewed texts and small fonts, requiring no fine-tuning.
 - Robust and flexible across diverse document layouts, countering the brittleness to layout sensitivity in VLMs.
 - Prioritizes information retrieval (IR) over unnecessary layout understanding as in reality, government documents lack reading sequences and are used for direct information extraction aligning with IR principles, improving accuracy.
- **IVAN-Translator Component:**
 - Customized algorithm combining PolyGlott for source language detection and Facebook's nllb-200-distilled-600M for translation, enabling the domain shift to English-text space, maintaining low complexity.
 - Handles code-mixed texts from OCR and ASR, providing robustness to noise, spelling errors and hallucinations.
- **IVAN-NER Component:**
 - Trained on noisy OCR-derived document texts, enabling generalization on semantically sparse document outputs.
- **The GovAI-Sub Web-App:**
 - Simplifies and fully automates complex government form-filling through a simplified, easy-to-interact user-interface incorporating an intuitive end-to-end AI pipeline, ensuring accessibility across diverse populations and language groups.

TRAINING WORKS UNDERTAKEN

During the course of this project, we gained valuable hands-on experience in developing an AI-based web application under the guidance and resources provided by Intel. Firstly we were introduced to the WSL (Windows Subsystem for Linux) environment and Python Virtual Environments which served as the primary platforms for our development and deployment activities. Secondly we came to know about various AI tools and mechanisms and the current AI market for them, like LLMs, VLMs, RAG, LangChain, Prompt Engineering, Prompt Writing, using AI tools like Chat Bots, Machine Learning, Language Modelling, various Python tools used in AI, etc. We also came to know about the current market for Language based tasks like NLP and the current market shift from traditional NLP Processes to Modern Deep Learning based NLP Processes and Models like LLMs, Transformers, Encoders, etc. Then we came to know about two most important cloud based platforms, GitHub and Hugging Face for deployment and AI modelling. We were introduced to various state-of-the-art AI models used in Vision and Language or in combination, like the Llama Model, GPT Model, BERT Model, Paddle OCR, Easy OCR, etc. We were also provided with quality resources and study materials on these topics and various other AI based materials from Intel during this internship period.

As part of the implementation, we learned to use Django, a Python-based web framework, for designing the backend structure, integrating AI models, and managing database operations. As our backend database for our web app, we learnt to use SQLite along with HTML, CSS and JavaScript for UI designing. We explored various artificial intelligence and natural language processing models, especially dedicated for Indic script supports such as Optical Character Recognition (OCR) using PaddleOCR and Surya OCR, Automatic Speech Recognition (ASR) using OpenAI Whisper Large-v3 module, Machine Translation using Facebook's "No Language Left Behind" initiative's NLLB-200 Distilled 600M model along with Polyglot Framework for language auto detection, Named Entity Recognition (NER) using fine tuned version of BERT-large-NER model from Hugging Face by "dslim" and postprocessing strategies using Regex. Various model optimization techniques were critically worked on by us during the course of this project. We also came across various validation strategies and metrics used in NLP domain like "n-grams" and "bag-of-words" based strategies, span-based and token based evaluation of NER models, WER and CER metrics common in OCR evaluation, F1 Score based evaluation adaptation for OCR, BERTScore, Edit Distance, Normalized Levenshtein Distance and many more.

Through this project, we developed a strong understanding of multilingual user interface design, AI model integration and workflow automation, gaining practical insight into how AI can be applied effectively to simplify real-world citizen service processes.

RELATED WORK

The evolution of automated form-filling has transitioned from basic optical character recognition and static rule-based systems to sophisticated multimodal document intelligence. This section provides a technical analysis of existing research and establishes the comparative necessity of the IVAN AI framework.

Analysis of Previous Works: Research in accessibility-focused automation often prioritizes specific interaction modalities. Hegde et al. [6] introduced the Dhvani voice Bot, utilizing the RASA framework and Knowledge Graphs to assist monolingual Kannada speakers with government schemes. While achieving 97% intent accuracy, the system is constrained by its lack of document processing integration and reliance on a single regional language. In contrast, multimodal efforts like those by Syed [29] explore the integration of voice and image recognition, yet often lack a cohesive pipeline for the linguistic diversity of the Indian subcontinent. The Vistaar benchmark [27] further underscores this gap, revealing that while ASR for 59 Indian languages is maturing with reduced Word Error Rates (WER), its integration into downstream automation workflows remains largely unexplored.

The domain of Optical Character Recognition (OCR) and Information Extraction (IE) has seen significant attempts at handling structured data. Traditional pipelines [13], [15], [30] have focused on identity documents like Aadhaar and PAN cards, utilizing PyTesseract or Google Vision APIs for direct field extraction from images. However, some of these systems often rely on static Regex-based methods of IE and post-processing that fail under the noise of poor-quality scans, stamps or complex varying layouts [12]. Lat and Jawahar [7] addressed some of these concerns through Deep Learning based preprocessing using their Super-Resolution framework, achieving a 21.19% accuracy gain, but their work remains English-centric and sensitive to cluttered layouts. Recent benchmarks such as Clutter OCR Bench [12] and PreP-OCR [11] highlight that even state-of-the-art models suffer from high Character Error rates (CER) of over 55% when faced with physical complexities and clutters like ink spills or food stains, complex layout distortions and most specifically code-mixed scripts common in real-world governmental documents.

The rise of Vision Language Models (VLMs) and Generative Information Extraction (GIE) [17] has introduced layout-aware understanding. Modern architectures like SCAN [10] and zero-shot Table-of-Contents guided RAG frameworks [20] outperform traditional OCR in spatial reasoning and textual retrieval. However, as noted in FormFactory [26] and surveys by Ghosh et al. [19], these models act as "black-boxes" with high computational overhead, making them unsuitable for edge deployment. Semantic document analysis using Graph

Convolutional Networks (GCN) [33] or Bayesian Networks [28] has shown success in predicting form fields but requires massive historical datasets and often excludes unstructured textual fields. Furthermore, while AI-powered solutions like DocuMind [23] and Skyvern [25] offer high efficiency, their reliance on expensive, cloud-based APIs like GPT-4, Amazon Bedrock or AWS Textract [22] hinders offline use, while raising critical data privacy and latency concerns for sensitive public-sector data [16], [21], [24].

Comparison and Countering of Existing Challenges:

Technical Aspects	Current Research	IVAN AI
Multilingualism (22+ Indic Languages)	✗ [6], [13]–[15], [22], [27]	✓
Multimodality (OCR & ASR)	✗ [6], [17], [26], [29], [30]	✓
Modularity (Distributed Glass-box Pipeline)	✗ [16], [17], [21], [26]	✓
Cost-effective and Open-Source (no Cloud-Service Costs)	✗ [16], [21]–[26]	✓
Lightweight & Edge-Ready (Low-Resource & On-Premise Deployment)	✗ [16], [19]–[23]	✓
Simplified Data Requirements	✗ [6], [8]–[10], [13]–[25], [27]–[33]	✓
Image-Space to English-Text-Space (Domain Shift)	✗ [16], [17], [19]–[21], [33]	✓
Code-Mixing Handled (Mixed Scripts)	✗ [7], [8], [10]	✓
Immunity to Changing Layouts (non-Layout centric)	✗ [10], [18]–[20], [32], [34]	✓
Immunity to Uneven Document Texts (Skewed)	✗ [11]–[14], [16]–[19], [34]	✓
Immunity to Noise & Hallucination (OCR-ASR-Translator-NER)	✗ [7]–[15], [32]	✓
Cross-Document Generalization (9 Types)	✗ [6], [9], [13], [15], [23], [28]	✓

Comprehensive Benchmarking	✗ [7], [10], [18], [20], [26], [27], [30], [34]	✓
Simplified and digitalized form-filling process	✗ [1]–[5]	✓
Wide-scale Adaptability and Accessibility	✗ [6]–[10], [13]–[20], [21]–[30], [32], [34]	✓

Table 1: Comparative Analysis of Existing Works vs. IVAN AI & GovAI-Sub

In summary, while prior works have significantly solved numerous challenges of automating form-filling systems in India with great precision, IVAN AI and GovAI-Sub are the first to address significantly persistent challenges and synthesize these into a robust noise-resilient, multilingual, multimodal, light-weight, modular and accessibility-first pipeline specifically optimized for the Indian socio-technical context and script limitations with practical benchmarks.

DATASETS

Phase-I (Component Specific Datasets)

- **OCR Dataset:** Collected various personal Documents like Aadhaar Card, Pan Card, Bank Passbooks, etc., from friends, family and self, manually extracted the text from them to make a test dataset for OCR result evaluation using Ground Truth Analysis.
- **ASR Dataset:** Collected various types of audio data of varying duration, ascent and languages (even code-mixed) and extracted the text from them for ASR evaluation using Ground Truth Analysis.
- **Translator Dataset:** Prepared a small test dataset having text in multiple languages with defined source and target languages, using it for evaluating Translation and Language-Auto Detection models using Ground Truth Analysis in both English–Indic and Indic–English directions.
- **IVAN-NER Dataset:** Derived text information from various government documents (9 docs) in suitable reading order of information and replicated them to make a small text dataset of 100 rows. Then annotated them manually with Label Studio to make a BIO Format CoNLL Style NER dataset for fine tuning and evaluating IVAN-NER Model, with few shot training to generalize across 21 form specific entities.

Phase-II (Pipeline Specific Datasets)

- **OCR-NER-Pipeline Dataset:** Document images with extracted ground-truth texts from them (code-mixed), source language and target English language labels with the actual translated English text and tagged entity lists from those texts were derived to make the dataset for evaluating the OCR-Translator-NER Pipeline.
- **ASR-Translator-Pipeline Dataset:** Multilingual, code-mixed audio samples with ground-truth transcripts, source-target language labels and reference translations to frame the dataset for testing the ASR-Translator Pipeline.

NOTE: All the Data contained in the above Datasets belong to the members and relatives of the members of GENAI-2 group, and any relevance to any external individuals or entities is highly coincidental.

DEVELOPMENT ENVIRONMENT

The GovAI-Sub web-app was implemented locally in the VS Code environment, while the IVAN AI system was developed using a distributed cloud-based platform to accommodate the diverse deep learning frameworks and GPU (VRAM) requirements of the IVAN AI components, due to the lack of suitable hardware configuration on local machines. Thus, the pipeline was executed across Google Colab and Kaggle Kernels to utilize their GPU enable development environments.

- **VS Code Environment Setup:** The VS Code environment was setup with the WSL (Windows Sub-System for Linux) environment using the Ubuntu distribution on local machine, to facilitate better file and package (using the UV package manager) management, increase adaptability and create a clean, efficient development environment. The GovAI-Sub web-app was implemented in this setup using the Django framework inside a Python Virtual Environment to mitigate version conflicts.
- **Google Colab Environment setup:** Running Python 3 kernel, the utilized hardware configurations comprised of 12.7 GB System RAM, 107.7 GB Disk Space and a single NVIDIA T4 GPU with 15 GB VRAM. The Surya OCR, Whisper Large-v3, BERT-Large-NER, PolyGlott framework and the nllb-200-distilled-600M were implemented on this environment along with their validation on the test datasets.
- **Kaggle Environment setup:** Running Python 3 kernel too, utilized 30 GB System RAM, 57.6 GB Disk Space and dual NVIDIA T4 GPUs each of 15 GB VRAM. The PaddleOCR running on the PaddlePaddle Deep Learning framework was implemented in this environment, by stripping the environment off PyTorch, that is not supported by PaddleOCR components.

Aside the local VS Code environment, the minimal setups for the cloud platforms were available through the standard configurations of Colab and Kaggle, and were sufficient for complete IVAN AI implementation and evaluation, demonstrating the system's lightweight and resource-efficient design.

METHODOLOGY

Phase-I (Component Prototyping)

- **The OCR Module (IVAN-OCR):** To gain Indic code-mixed document support and robustness to diverse quality of document images, the Surya OCR [35] was implemented for processing the uploaded Documents from the User. All the scanned PDFs or Images of document are OCRed to extract the texts from documents, supporting code-mixed Indic scripts. This process was tested on two specially designed algorithms other than basic Surya OCR, by using its dedicated internal models for enhancing the OCR results on diverse complex document types.

- **Algorithm 1: Surya Layout-Detection-Recognition (SuryaLDR)**

// Layout Detection followed by Text Detection followed by Text Recognition

SuryaLDR(doc){ *// doc can be image/pdf*

Step 1: input $\leftarrow$ Image(doc)

Step 2: layouts $\leftarrow$ LayoutPredictor(input, pretrained_weights)

Step 3: for section in layouts{

 bounds $\leftarrow$ section[bboxes]

 cropped_img $\leftarrow$ Crop(input, bounds)

 bbox $\leftarrow$ DetectionPredictor(cropped_img, pretrained_weights)

 text $\leftarrow$ RecognitionPredictor(cropped_img, bbox, pretrained_weights)

}

Step 4: return text

}

- **Algorithm 2: Surya Layout-Recognition (SuryaLR)**

// Layout Detection followed by Text Recognition, bypassing Text Detection

SuryaLR(doc){ *// doc can be image/pdf*

Step 1: input $\leftarrow$ Image(doc)

Step 2: layouts $\leftarrow$ LayoutPredictor(input, pretrained_weights)

Step 3: for section in layouts{

 bbox $\leftarrow$ section[bboxes]

 text $\leftarrow$ RecognitionPredictor(input, bbox, pretrained_weights)

}

Step 4: return text

}

The performance analysis of these two OCR algorithms along with base Surya OCR are shown in detail in Results and Discussions section further.

- **The ASR Module (IVAN-ASR):** For voice modality, the OpenAI Whisper Large-v3 [39] was employed to process the uploaded Audio data from the User. Unlike standard implementations, the model was configured with the "transcribe" mode rather than direct "translate" mode. This design choice preserves the nuances of code-mixed Indic speech and efficiency of the ASR model, which are later processed by the IVAN-Translator. This ensures higher semantic fidelity for regional dialects and mixed-language inputs common in the Indian demographic.
- **The Translator Module (IVAN-Translator):** The translation module serves as a bridge to the simplified English-centric processing domain post-OCR and Indic form-filling post-ASR. The PolyGlott framework [37] was utilized for high-speed automatic language detection across code-mixed inputs. The detected source language(s) then gets passed to the Facebook nllb-200-distilled-600M [38] model along with the input text and target language for translation. The code-mixed text from OCR is translated to English (if not already) and transcribed ASR text is translated into the language selected by the user for form-filling using the IVAN-Translator module, serving as the key to simplify the handling of multilingual, code-mixed strings by translating them into a unified (English or Indic) text space. The "distilled" variant of NLLB was selected to maintain a lightweight footprint and repeated fallbacks in the pipeline.

- **Algorithm 3: IVAN-Translator (PolyGlott-NLLB)**

// Language auto-detection followed by Machine Translation

```

IVAN_Translator(text, target_lang){ // text from ASR or OCR
    Step 1: input ← text
    Step 2: source_lang ← Detector(input)
    Step 3: model ← AutoModel(translator)
    Step 4: translated ← model.generate(input, source_lang, target_lang)
    Step 5: return translated
}

```

For voice modality, these translated values are directly mapped to the appropriate Form Fields, while for document uploads the final translated English text is fed into the IVAN-NER module for form-entity extraction, to accomplish the Auto-Filling process.

- **The NER Module (IVAN-NER):** For form-entity extraction from OCR text, a domain-adapted BERT-Large model [40] was implemented, pre-trained on the CoNLL-2003 dataset [41]. The model was further fine-tuned on a domain-specific dataset (IVAN-NER Dataset) in an 80:20 split to produce the BERT-Doc-NER model, generalizing across 21 government specific entities and 9 government document types. The architecture utilized a 24-Layer Encoder-based Deep Neural Network, a Linear

Activation layer as classifier and GELU as hidden-activation, while the training setup used the AdamW as the optimizer, LambdaLR scheduler with linear warmup, with a weight decay of 0.03 and initial learning rate of 3e-5. Training was configured for 10 epochs with early Stopping mechanism monitoring validation loss, that terminated the process at epoch 8 preventing overfitting.

Phase-II (Enhancements and Integration)

- **The OCR Module Enhancements (IVAN-OCR):** To address the limitations of monolithic OCR engines, specifically poor text detection results of Surya-OCR [35] from Phase-I, the PaddleOCR [36] known for its robust text detection capabilities from complex images was implemented. But for the high computational cost of end-to-end PaddleOCR models on complex Indian documents, cross-engine hybrid OCR architectures were developed using the lightweight, dedicated modules of PaddleOCR with the Recognition module of Surya OCR in order to optimize the OCR system on complex degraded government documents, while being lightweight.

- **Algorithm 4: Hybrid Paddle-Layout Surya-Recognition (HybridLR)**

// Paddle's Layout Detection followed by Surya's Text Recognition

HybridLR(doc){ *// doc can be image/pdf*

Step 1: input $\leftarrow$ Image(doc)

Step 2: model $\leftarrow$ LayoutAnalysis(layout_detector)

Step 3: img, bboxes $\leftarrow$ model.predict(input)

Step 4: text $\leftarrow$ RecognitionPredictor(img, bboxes, pretrained_weights)

Step 5: return text

}

- **Algorithm 5: Hybrid Paddle-Detection Surya-Recognition (HybridDR)**

// Paddle's Text Detection followed by Surya's Text Recognition

HybridDR(doc){ *// doc can be image/pdf*

Step 1: input $\leftarrow$ Image(doc)

Step 2: model $\leftarrow$ TextDetection(text_detector)

Step 3: img, bboxes $\leftarrow$ model.predict(input)

Step 4: text $\leftarrow$ RecognitionPredictor(img, bboxes, pretrained_weights)

Step 5: return text

}

To mitigate the environmental conflicts between the PaddleOCR (build on PaddlePaddle) and the Surya OCR (build on PyTorch), a serialized Pickle buffer was used to store the layout or text detection results from PaddleOCR (bounding-boxes or bboxes) which was then shared for text recognition using Surya OCR.

- **The NER Module Enhancements (IVAN-NER):** From the training and performances of Phase-I, the BERT-Doc-NER trained on 80:20 split was additionally fine-tuned again on the full IVAN-NER Dataset of 100 samples to enhance pattern generalization and observe NER model improvements to data augmentation, producing the BERT-Doc-NER-Max model. The model in Phase-II was fine-tuned with the same model configurations and training parameters as in Phase-I, but for 5 epochs.
- **OCR-Translator-NER Pipeline (Document Input):** This pipeline is designed to automate form-filling tasks by processing uploaded document image or PDF and extracting key information from them for filling forms.
 - **Stage 1:** The uploaded document image or PDF is processed by IVAN-OCR to extract raw, multilingual text blocks from them.
 - **Stage 2:** IVAN-Translator auto-detects the source language(s) from the OCR-text and converts them into a standardized English format.
 - **Stage 3:** The English texts from Stage 2 is fed into the IVAN-NER model to extract the relevant target entities for the process, among the trained set of 21 entities in a JSON format, suitable for filling in forms.
 - **Stage 4:** The form specific entities derived in Stage 3 as JSON are re-translated by the IVAN-Translator from English to the language in which the user is filling the form in and directly mapped to form-fields.

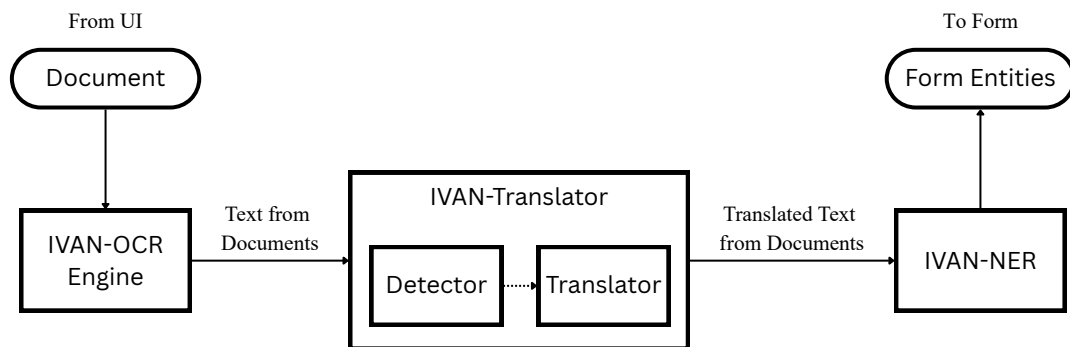


Figure 1: OCR-Translator-NER Pipeline

- **ASR-Translator Pipeline (Voice Input):** Designed for accessibility, this pipeline automates form-filling tasks by processing user information as input speech. By soliciting direct field-specific information from the user, this pipeline bypasses the NER stage reducing complexity (e.g., "Speak your Aadhaar number").
 - **Stage 1:** Real-time audio information is captured and transcribed by IVAN-ASR into raw text as JSON.
 - **Stage 2:** The transcribed ASR-text in JSON format (potentially code-mixed) is translated by IVAN-Translator into the target language of the form, producing a translated JSON output.

- **Stage 3:** The translated information from Stage 2 is directly mapped to the corresponding form fields.

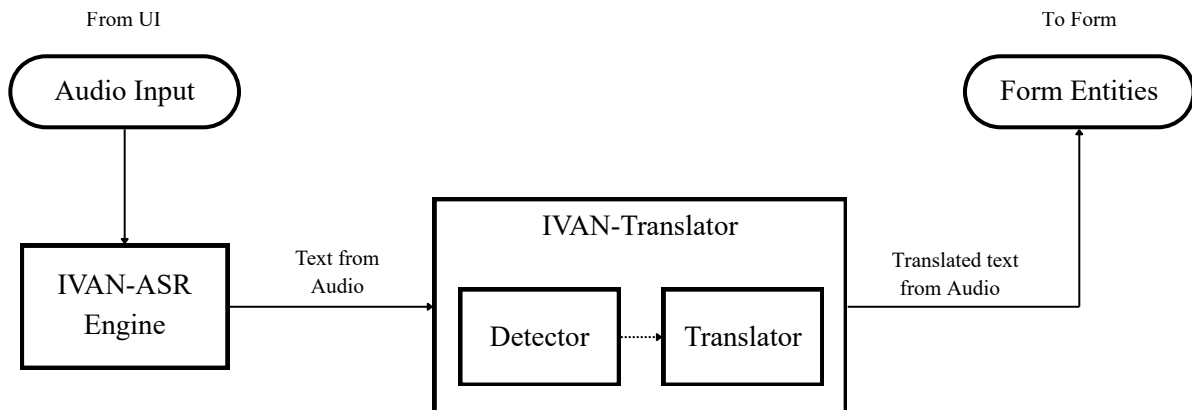


Figure 2: ASR-Translator Pipeline

- **The GovAI-Sub Web App:** The mitigation of current challenges in governmental procedures, mainly due to the language, literacy, physical, digital and other challenges requires simplification of the governmental processes in the form of a digitalized platform that can be easily accessed and used by a large population of Indian citizens across these barriers. The GovAI-Sub is made with these views in mind, implemented using the Django framework [43], with HTML, CSS and JavaScript [44]–[46] for the UI, the Interface aims to provide users with an easy-to-use, user-friendly, multimodal, multilingual, automated, digital platform with easy and clear set of instructions to complete government forms of all types, in any supported language with real-time submissions to government portals without delays, all from a single place. The web app also supports offline downloads for manual processing of the forms if needed, making GovAI-Sub a highly useful and widely accessible and adaptable digital solution to the complex, laborious and time-consuming procedures of traditional governmental frameworks, highly aligning with the Digital India Initiatives [3], [4].
 - **Stage 1 (Public Home Page):** Users are prompted to register with basic details like name, email, phone no. and password, or to login if existing user.
 - **Stage 2 (Private Home Page):** Users are asked to choose their desired form to fill or prompted to complete any previously opened but incomplete form.
 - **Stage 3 (Language & Modality Page):** Users are prompted to choose their preferred form language (currently supports English, Hindi, Bengali, Marathi, Tamil, Telugu) and mode of operation among Document or Voice input (Handwritten mode of input is not supported yet). Then they are asked to provide the necessary information as Document or Voice and the provided data is processed by the OCR-Translator-NER pipeline or the ASR-Translator pipeline accordingly and stored in a Temporary Virtual Buffer. The processed information is then auto-filled to the corresponding form fields in the user's chosen language.

- **Stage 4 (Review, Edit and Submit Page):** The user is shown an editable preview of the auto-filled form, where he/she can review the form and make any corrections needed and once satisfied, the form can be submitted and the details gets saved in the Database for reuse in future.
- **Stage 5 (Submission History Page):** The user can view and download their submission details as PDF for further manual processing.
- **Stage 6 (Profile Page):** The basic details of the user uploaded during form filling processes can be viewed on the Profile Section, accessible from the Home (Private Home) Page.

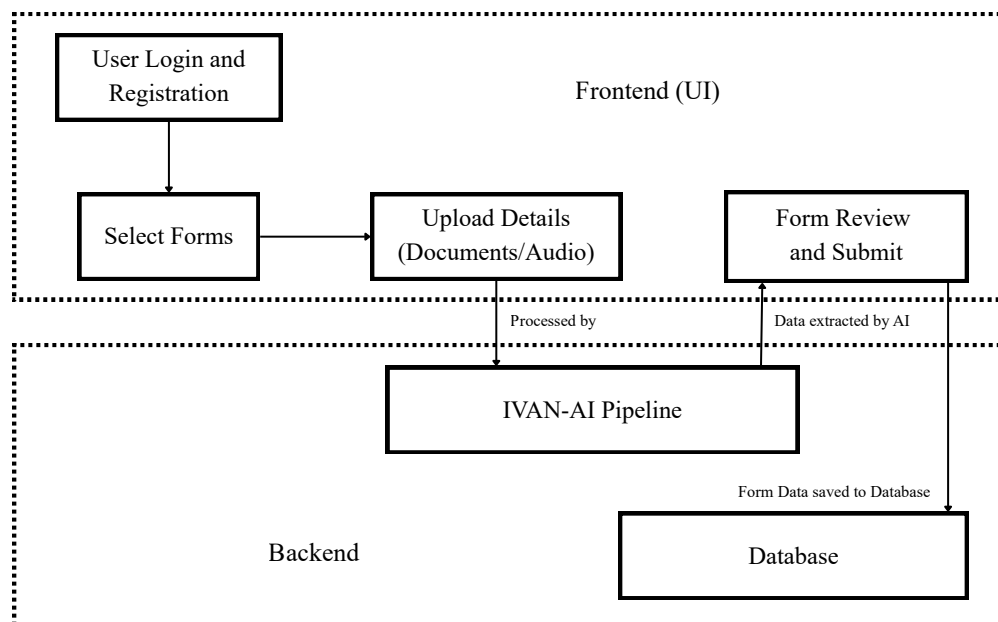


Figure 3: GovAI-Sub Web App

SYSTEM HIGHLIGHTS

Key Features of the System: The additional features of the GovAI-Sub web app and the IVAN AI are highlighted in the following sections of the report.

- **GovAI-Sub Web App:**

- **Session Management:** The Web App supports Session Management where each new Login creates a new Session and each Logout terminates a Session successfully. User data is saved session wise in the Database which helps to keep track of user activity.
- **Crash Management:** The Web App is made immune to Database or Server crashes, where any scenario of an unstable Database state, where an user has opened a form for submission but was not able to submit it in that Session due to intentional or forced Logout actions (considered as a faulty Session), is auto Rolled back to regain stability.
- **Database Schema (Relver Details):**
 - Users (user_id, name, email_id, phone_no, password, dob, is_active, picture)
 - UserAddress (address_id, current_address, permanent_address)
 - PensionDetails (pension_id, joining_date, leaving_date, reason_of_leaving, nominee, relationship)
 - PersonalInfo (info_id, birth_place, gender, aadhaar_no, pan_no, father, mother, spouse, qualification, occupation)
 - BankDetails (bank_detail_id, bank_name, cif, account_no, ifsc)
 - Forms (form_id, languages, description, proformas, category)
 - Sessions (session_id, form_language, submission_date_time)
- **Session & Crash Management Logic:**
 - if users.is_active = 0 and (sessions.form_id = null and sessions.submission_date_time = null), then
→ no form-filling Session (no Rollback needed) *// system in stable state*
 - if users.is_active = 0 and (sessions.form_id = not null and sessions.submission_date_time = null), then
→ faulty Session (Rollback needed) *// system in unstable state*
- **Virtual Buffer Management:** In order to handle the auto Session Rollbacks in case of faulty Sessions (intentional or forced), the semi-uploaded user data (document images or spoken information) from that form-filling Session is saved in a virtually maintained buffer for a limited amount of time (temporary), so that the user is offered a chance to complete or close the already opened form from that exact state, before opening a new one in the next Session, even though the Database does not hold its details any longer.

◦ Entity Relationship (ER) Model for the Database:

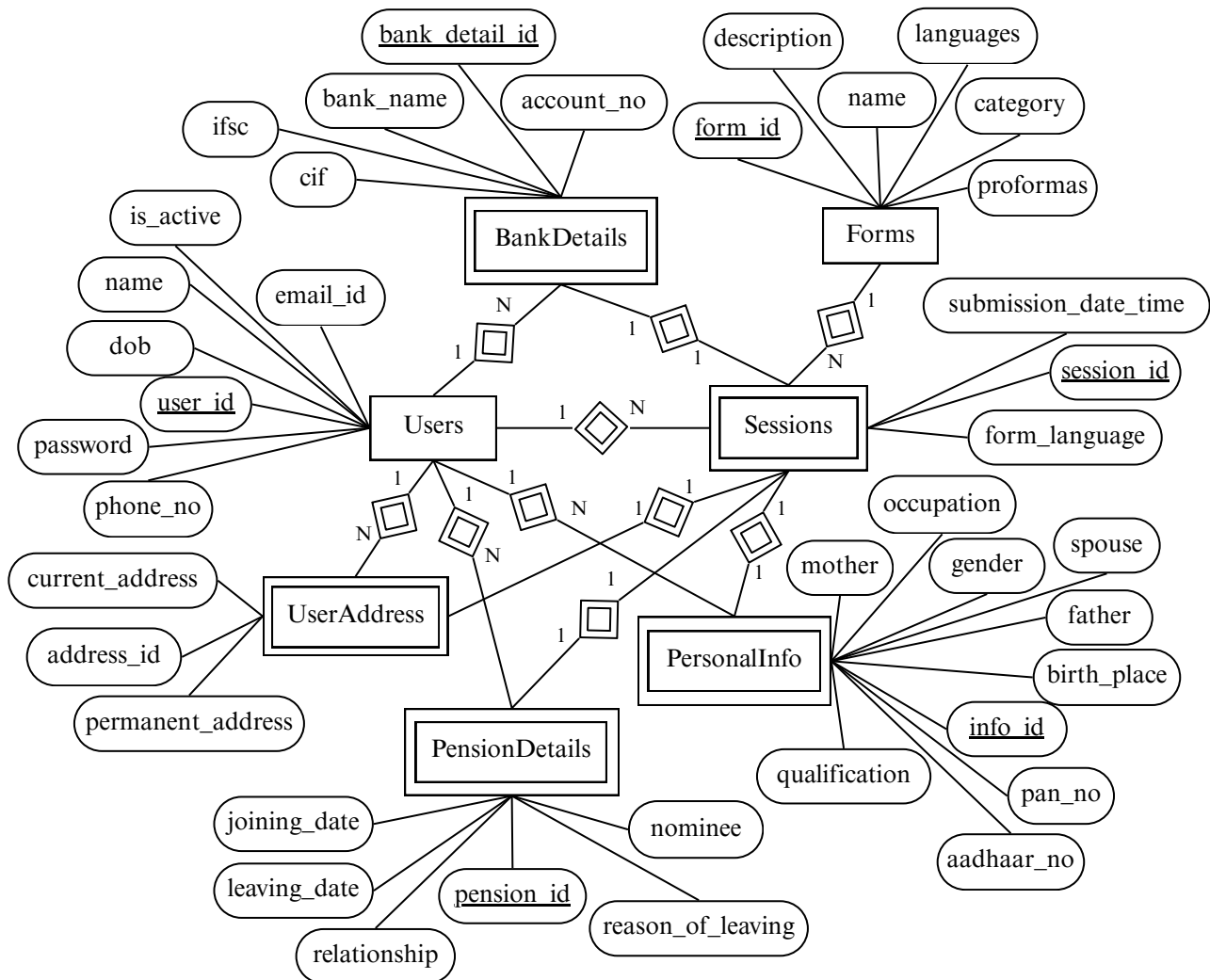


Figure 4: ER-Diagram

• IVAN AI Framework:

- **Multilingual Support:** The underlying components of IVAN AI support diverse set of Indic languages, with the Paddle-Surya hybrid approaches supporting 21 Indic languages, IVAN-Translator (PolyGlott & nllb-200-distilled-600M) supports 22 and IVAN-ASR (Whisper Large-v3) support 12 Indic languages.
- **IVAN-OCR Reading Order (Phase-II):** In Phase-II implementation, a standard top-bottom left-right reading order is followed by the OCR algorithms.

The overall System of GovAI-Sub & IVAN AI has been designed to be lightweight and user friendly, with the aim of deploying this end-to-end framework on edge devices like smart-phones, tablets, etc. in future, supporting offline use and enhance accessibility.

System Design Constraints: For simplicity of implementation and due to lack of specific computational resources, the GovAI-Sub and the IVAN AI systems inherit certain temporary designing and processing constraints that can be re-designed in future for real-life deployment.

- **The GovAI-Sub Web App:**

- GovAI-Sub allows submission of single forms per Session and the user gets auto logged-out, terminating the Session after one successful submission.
- GovAI-Sub Web App is not integrated with IVAN AI due to the lack of GPU enabled local-Systems. Thus the AI based Auto Form-Filling feature though present, is currently inactive in the Web Interface.
- GovAI-Sub doesn't support multilingualism in any aspects till now due to the lack of integrated IVAN-Translator Model for this purpose.
- The Virtual Buffer implemented in GovAI-Sub is currently inactive as no AI pipeline is there to provide processed data for the Buffer to store.
- GovAI-Sub does not support direct online submission of forms to Government portals like DigiLocker, due to the lack of legal access to such government APIs.

- **The IVAN AI Framework:**

- IVAN-OCR has not been designed specifically and has not been tested on Handwritten scripts for information upload, so performance in this regard is not guaranteed. Though it might perform good for certain levels of handwriting, but proper architecture designing is needed for detecting diverse varieties of complex handwritten data.

RESULTS AND DISCUSSIONS

Evaluation Background (for both Phase-I and Phase-II):

- The validation and evaluation of the IVAN AI components and IVAN AI Pipelines were done on the pre-defined Colab and Kaggle environments with the same setup.
- While the underlying models of the Paddle-Surya hybrid, nllb-200-distilled-600M and Whisper support 21, 22 and 12 Indic languages respectively, the empirical validation for IVAN AI was conducted on six representative languages, English, Bengali, Hindi, Marathi, Tamil and Telugu. This subset was selected to represent diverse script families (Indo-Aryan and Dravidian) while maintaining a manageable scope for rigorous ground-truth verification.
- The system validation was deliberately restricted to 9 document types and 3 synthetic forms to prevent inadvertent creation of unauthorized digital mirrors of official government portals and to keep the evaluation process on a manageable-scale. By validating on synthetic forms that replicate the structural complexity of actual government forms including technical terminology, nested fields and multi-column layouts, the system's technical capability is demonstrated accurately while maintaining compliance with legal and ethical development standards.

Phase-I (Component Analysis)

Each individual component of the IVAN AI Module was tested separately whose report is shown below, full IVAN AI Pipeline testing is done in Phase-II.

• IVAN-OCR Benchmarks:

Model	Text Extraction (%)	Evaluation Metric	Time (s)
Surya OCR	56.8	Normalized Levenshtein Dist. (Acc.)	5.43
Surya LDR	41.4	Normalized Levenshtein Dist. (Acc.)	30.9
Surya LR	59.02	Normalized Levenshtein Dist. (Acc.)	17.3

Table 2: IVAN-OCR Benchmarks

- The Custom Surya LR OCR Algorithm (59.02%) outperforms normal Surya OCR (56.8%) in token-order based evaluation of text extraction from documents, but with a bit higher processing time (17.3s).
- This shows that proper and accurate text detection (or bounding-box detection) can significantly enhance the overall text extraction performance of OCR systems.
- Thus in Phase-I, we select the Surya LR Algorithm as the IVAN-OCR Model, but text-detection still performs poorly with Surya LR on complex document layouts.

• **IVAN-ASR Benchmarks:**

Model	Transcription (%)	Evaluation Metric	Time (s)
IVAN-ASR	73.7	Normalized Levenshtein Dist. (Acc.)	3.02

Table 3: IVAN-ASR Benchmarks

- The integration of OpenAI Whisper Large-v3 for ASR gives significantly good results for multilingual code-mixed text transcription across diverse test cases.
- However, persistent challenges involving complex ascents, limited training for low-resource languages, high background noise and its inefficiency in handling numeric strings hampers transcription and proper language detection performances.

• **IVAN-Translator Benchmarks:**

Model	Translation (%)	Language Detection (%)	Overall (%)	Evaluation Metric	Time (s)
IVAN-Translator	69.9	25	47.4	Mean Aggregate of Detection (custom Penalty-based) & Translation (Normalized Levenshtein Dist. (Acc.))	0.297

Table 4: IVAN-Translator Benchmarks

- The NLLB-200-Distilled-600M model achieves high translation accuracy with Normalized Levenshtein Distance (Acc.) despite code-mixing, but PolyGlott's poor language auto-detection performance on the Customized Penalty based metric significantly reduces the overall IVAN-Translator pipeline efficiency.

- However, occasional challenges exist with IVAN-Translator with respect to poor training on low-resource languages though nllb-200-distilled-600M is immune to hallucinations, outdated nature of the PolyGlott library and low support for numeric strings of both nllb-200-distilled-600M and PolyGlott frameworks.

- **IVAN-NER Benchmarks:**

Model	Entity Extraction (%)	Evaluation Metric	Time (s)
IVAN-NER	73.49	F1 Score	0.062

Table 5: IVAN-NER Benchmarks

- The Model Performance on Validation data is significantly high with 73.49% F1 Score and a Loss of 0.419 with significantly low entity extraction time, indicating good generalization to unseen data. It captures the patterns of diverse varieties of document texts from the dataset and the diverse form entities present in them.
- But the size of the IVAN-NER Dataset is still quite small for proper generalization of an LLM across all of the 21 form entities and 9 difference document types, requiring data augmentation in future for better performance.

NOTE: The Normalized Levenshtein Distance (Acc.), also called as Normalized Similarity used in the evaluation of OCR, Translator and ASR is calculated as,

$$\rightarrow \text{Normalized Dist(actual, pred)} = \text{Dist(actual, pred)} / \max(\text{len(actual)}, \text{len(pred)})$$

$$\rightarrow \text{Normalized Acc(actual, pred)} = (1 - (\text{Normalized Dist(actual, pred)})) \times 100$$

Phase-II (Enhancements and Pipeline Analysis)

Phase-II focuses on evaluating the enhancements made in IVAN-OCR and IVAN-NER components, evaluating IVAN AI components on different metrics as from Phase-I with respect to their unique strategy of performance in IVAN AI framework, followed by model selection as per the modified component performances from both the Phases. Based on the most suitable alternatives from model selection process, the end-to-end OCR-Translator-NER and ASR-Translator Pipelines are framed and evaluated.

- **Metric Choice and Hypothesis:** Evaluation of the IVAN AI system required a paradigm shift from traditional OCR and speech processing metrics that prioritize sequential, layout-dependent accuracies. Conventional benchmarks like Word Error

Rate (WER) and Character Error Rate (CER) assume a linear, accurate reading order, however Indian government documents such as Aadhaar, PAN, Birth Certificates, etc., often lack a predefined semantic sequence. Because these documents are utilized for discrete data extraction rather than continuous reading, the system adopts the principles of Information Retrieval (IR). This approach prioritizes degree of retention of key details from document scans over spatial arrangement, ensuring critical information is preserved for NER extraction regardless of document layout or noise.

- **Metric for IVAN-OCR:** F1-score is designed as the primary metric for evaluating enhanced IVAN-OCR from Phase-II, computed using a “bag-of-words” strategy, assessing overlap between predicted and ground-truth information sets. This emphasizes on semantic completeness, whether essential entities like names, identification numbers, addresses, etc., are correctly extracted rather than penalizing the system for token order variations inherent in diverse government documents. Normalized Similarity (or Normalized Levenshtein Distance (Acc.)) scores are reported as supplementary metrics for comparative analysis with Surya LR from Phase-I.
- **Metric for IVAN-ASR:** The standard metric used is to re-evaluate IVAN-ASR from Phase-I is BERTScore, that evaluates semantic similarity between transcribed speech and ground-truth text through precision, recall and F1-score using contextual embeddings. This prioritizes semantic equivalence, even without exact lexical matches essential for voice-based form-filling where users might articulate information with varied linguistic structures.
- **Metric for IVAN-Translator:** This employs BERTScore-based metrics to quantify semantic fidelity of translated text. Given multilingual and code-mixed Indian documents, exact word-for-word translation is infeasible, and focus remains on preserving informational semantics and integrity. Language detection accuracy uses “bag-of-word” based F1-score, similar to IVAN-OCR accounting for correct predictions, penalizing for misclassifications and undetected languages. Overall translation score averages detection and translation F1-scores.
- **Metric for IVAN-NER:** Used entity-level F1-score as standard metric and validation-loss (for training evaluation) within the BIO tagging scheme as in Phase-I, reflecting the system's ability to extract structured information from noisy OCR-like text. Inference time per sample assesses deployment feasibility in resource-constrained environments.

These metric choices aim to prioritize semantic accuracy, information retention and practical deployment over traditional layout-centric or token-level error measures, aligning with the functional requirements of government form automation domain where accurate field population is paramount.

- **Case Study–How common citizens actually interpret Government Documents (Metric Choice Justification):** A small user study was conducted with two volunteers, Volunteer 1 and Volunteer 2 to observe how people actually read, interpret and extract information from government documents while filling forms.
 - **Test Setup:** Both the volunteers were asked to fill a particular target form from our set of 3 synthetically designed forms, needing information from two government documents, a pension service book and a bank passbook.
 - **Task:** The volunteers were asked to do a simple task, “Fill the given form with the help of these two documents.”, and their approach of deriving information from the documents were noted and based on that the study was performed.
 - **Volunteer 1:** Extracted all the necessary information sequentially from the document in a rough key-value pair format.
 - **Pension Service Book:** Unaware of this document before, volunteer 1 needed a rough reading of the document to locate key information in it.
 - **Bank Passbook:** Already aware of this common bank document, volunteer 1 quickly spotted the information needed, without any dedicated pre-reading.
 - **Volunteer 2:** Completed each form-field sequentially by directly extracting the information needed from the respective document before moving to the next field.
 - **Pension Service Book:** Volunteer 2 already aware of this document type needed no dedicated reading for extracting necessary information.
 - **Bank Passbook:** Also aware of this document, volunteer 2 needed no dedicated reading for deriving necessary information.
 - **Evaluation:** The proximity of how they processed the documents was evaluated using Word Error Rate (WER) between volunteer 1 vs. volunteer 2.
 - **Results:** The WER values from the experimentation were quite high,
 - **Pension Service Book:** Showed a WER of 98.23%.
 - **Bank Passbook:** Showed a WER of 96.8%.
 - **Conclusion:** The high WER values show significant dissimilarity in how different users interpret government documents and that the interpretation, usage and reading sequence might change from user to user, based on the needs and knowledge. This provides strong evidence in support of our claim that “Government Documents have no predefined reading order, making traditional token-order based metrics for OCR invalid in this purpose”. Also the behavioral similarity of the volunteers in direct extraction of information from documents without reading them unless needed proves that “Government Documents are used for direct information retrieval without any sequential reading”.

This case study validates our choice of IR based evaluation for IVAN-OCR on government documents using F1 Score as the standard metric instead of traditional WER, CER or reading-order based evaluation metrics for OCR.

- **IVAN-OCR Benchmarks:**

Model	f1-score	Normalized Levenshtein Dist. (Acc.)	Time (s)
Hybrid LR OCR	62.24	50.323	8.03
Hybrid DR OCR	77.81	60.007	3.46

Table 6: IVAN-OCR Benchmarks

- Hybrid DR OCR outperforms Hybrid LR OCR in Phase-II in terms of F1 Score, Normalized Similarity and processing time. Also, Hybrid DR OCR (60.007%) in Phase-II outperforms Surya LR OCR (59.02%) from Phase-I, thus Hybrid DR OCR Algorithm qualifies in the model selection process for the IVAN-OCR component for designing the OCR-Translator-NER Pipeline.
- The IVAN-OCR (Hybrid DR OCR) maintains improved performance at IR-based evaluation (F1 Score) with a zero-shot, lightweight-hybrid architecture showing its efficiency in retention of key information from documents during OCR. Furthermore, the processing latency of 3.46s per page with its ability to handle old-degraded documents, skewed, code-mixed, multilingual texts enables real-time deployment, offering a high-efficiency alternative to computationally expensive and layout-sensitive VLM-based or cloud-dependent pipelines.

- **IVAN-ASR Benchmarks:**

Model	f1-score	precision	recall	Time (s)
IVAN-ASR	92.33	91.83	92.87	3.02

Table 7: IVAN-ASR Benchmarks

- IVAN-ASR achieves a BERTScore (F1 Score) of 92.33%, demonstrating high semantic fidelity in transcribing spoken data, which is far better than the token-order based Similarity Score of Phase-I, validating the superiority of semantics-based evaluation for IVAN-ASR.
- The high semantic-level performance across 12 Indic language support with 3.02s processing latency, makes it acceptable for asynchronous form-filling applications where ensuring the precision of transcribed legal entities is the primary priority.

Model	f1-score	det f1_score	precision	recall	overall score	Time (s)
IVAN-Translator	94.78	78.33	94.63	94.94	86.55	0.297

Table 8: IVAN-Translator Benchmarks

- The IVAN-Translator achieves 78.33% F1-score for language detection and 94.78% BERTScore (F1 Score) for translation, signifying the relevance of "bag-of-words" based and document-semantics based metrics for detection and translation evaluation respectively.
- The IVAN-Translator with a negligible latency of 0.297s while being immune to hallucinations, code-mixing and noise from OCR or ASR makes it a highly important component serving as a novel, lightweight, zero-shot framework for dataset limitations enabling the Domain-Shift strategy, also serving significantly in handling multilingual code-mixed scripts throughout the IVAN AI Pipeline.

• **IVAN-NER Benchmarks:**

Model	f1-score	Loss	Time (s)
IVAN-NER	79.8	0.1075	0.062

Table 9: IVAN-NER Benchmarks

- The IVAN-NER (BERT-Doc-ER-Max) after Phase-II re-training on a bit larger corpus of data, shows significant improvement in evaluation results of 79.8% F1 Score with a loss of 0.1075. Compared to Phase-I (73.49%), the accuracy increase in Phase-II is around 8.72%, showing that a minimal amount of data augmentation can significantly increase the IVAN-NER model performance.
- The BERT-Doc-NER-Max variant (from Phase-II) obviously surpasses BERT-Doc-NER (from Phase-I), thus being used in the OCR-Translator-NER Pipeline.
- The novel Domain-Shift strategy adapted in this project, that shifts the focus of data annotation and augmentation from Indic-Image based datasets to simple, resource-rich and easily replicable English-Text datasets significantly simplifies

future data augmentations, mitigating the challenges revolving around the dataset constraints and annotation difficulties in this domain, thus highly supporting proper NER model training and generalization for efficient form-automation.

- **OCR-Translator-NER Pipeline Benchmarks:**

Pipeline	OCR Score	Translator Score	NER Score	Overall Score	Time(s)
OCR-Translator-NER	77.81	86.77	79.8	81.46	6.33

Table 10: OCR-Translator-NER Pipeline Benchmarks

- This is the Document Input Pipeline that processes uploaded document Images and PDFs to extract key information from them for the automatic form-filling process.
- Overall Pipeline Score is the Mean Aggregate of IVAN-OCR (Hybrid DR), IVAN-Translator (PolyGlott & NLLB) and IVAN-NER (BERT-Doc-NER-Max) Scores.
 $\rightarrow \text{Pipeline Score} = (\text{OCR Score} + \text{Translator Score} + \text{NER Score}) / 3$

- **ASR-Translator Pipeline Benchmarks:**

Pipeline	ASR Score	Translator Score	Overall Score	Time(s)
ASR-Translator	92.33	76.37	84.35	3.027

Table 11: ASR-Translator Pipeline Benchmarks

- This is the Voice Input Pipeline that processes uploaded voice information to transcribe and translate them for the automatic form-filling process.
- Overall Pipeline Score is the Mean Aggregate of IVAN-ASR (Whisper Large-v3) and IVAN-Translator (PolyGlott & NLLB) Scores.
 $\rightarrow \text{Pipeline Score} = (\text{ASR Score} + \text{Translator Score}) / 2$

SPECIAL TERMINOLOGIES USED

Terminologies	Meaning
DetectionPredictor(...)	Text Line Detection Model of Surya OCR
RecognitionPredictor(...)	Text Recognition Model of Surya OCR
LayoutPredictor(...)	Layout Detection Model of Surya OCR
Image(...)	PIL Image Function
Crop(...)	Image.crop(...) function of PIL
bbox/bboxes	Bounding Box in images
Detector(...)	Language Detector Framework, currently PolyGlott
AutoModel(...)	AutoModelForSeq2SeqLM class from Hugging Face
translator	Translator Model, currently facebook/nllb-200-distilled-600M
TextDetection(...)	Text Line Detection Model of Paddle OCR
text_detector	Text Detection Model, currently PP-OCrv5_server_det
LayoutAnalysis(...)	Layout Analysis and Detection Model of Paddle OCR
layout_detector	Layout Analysis Model, currently PP-DocLayoutV2
BIO	Beginning-Inside-Out format for NER Datasets
CoNLL	A popular NER Dataset

Table 12: Special Terminologies

CONCLUSION

In Conclusion: This project presents the IVAN AI framework, integrable within the GovAI-Sub system, as a practical solution to India's digital governance challenges by enabling accessible, multilingual and multimodal automated form-filling for government services. Despite facing significant constraints from limited training data, legal restrictions on replicating government documents into a dataset with annotation complexities and various technical difficulties in developing IVAN AI pipelines on resource-limited environments, the system demonstrates considerable potential in bridging the digital divide for India's diverse population, with proved benchmarks. The hybrid approach of IVAN-OCR combining lightweight, zero-shot or few-shot models like PaddleOCR with robust engines like Surya-OCR for degraded documents, PolyGlott for language detection with NLLB-200 for translation and BERT-based entity extraction, alongside OpenAI Whisper for speech recognition effectively addresses the real-world complexities of Indian government forms featuring skewed layouts, multiple languages and varying document quality. While IVAN AI and GovAI-Sub integration was not successful due to computational constraints, the component-specific scores of the IVAN AI system and the successfully integrated end-to-end AI pipeline scores designed on those components mark significant progress and novelty in a domain where existing commercial solutions often fail on Indian systems due to their English-centric design, constrained resource environments or legal permissions. The easy-of-use, accessibility and adaptability focused design and development of the GovAI-Sub web interface also marks the significant mitigation of complex, time-consuming, manual governmental procedures, in the form of a fast-paced, digitalized platform. The system can process both voice and document inputs (dual-modality) in 17-22 Indian languages, pull out important information from unclear OCR or ASR transcribed text and automatically fill the correct form fields. This makes it easier for elderly people, individuals with disabilities, low literacy rates and rural communities to access government services without struggling through complicated paperwork, increasing accessibility and adaptability.

- **Drawbacks and Limitations:**

- IVAN-OCR does not follow any intensive image-preprocessing steps that might improve performance on highly degraded document images, working totally on raw images.
- IVAN-Translator occasionally suffers from low-resource languages, for numeric strings and from languages with fewer training examples during fine-tuning.
- The PolyGlott framework used for language detection is outdated and thus underperforms on the complex semantic nuances of modern languages.

- IVAN-ASR suffers from transcription errors in environments with high noise, on complex accents and on low-resource languages.
- The biggest constraint is the legal limitations in designing a large scale English-text dataset from OCR-derived government document texts for IVAN-NER fine-tuning, limiting the dataset size (100 samples), making future expansions possible only under legal permissions.
- The IVAN AI framework does not support extensive Handwritten document processing for information retrieval and form-automation.
- The actual automation of the form-filling was not possible to perform even though every component for the same is functional, due to GPU constraints on local VS Code environment.
- The GovAI-Sub web app does not allow the use of certain features (like Virtual Buffer, multilingual interface, form submissions to government portals, etc.) due to various underlying technical and legal restrictions.
- The overall system currently does not focus on any sort of privacy or security protocols in the pipeline for protecting sensitive user data, that is a serious factor in real world deployments.

Despite these challenges, the IVAN-AI and GovAI-Sub system provides a robust, end-to-end, modular and likely adaptable pipeline integrated successfully for automating Indic form-filing tasks with considerable benchmarks and novelties for being developed on such a complex, resource-constrained NLP domain. It counters significant research gaps, with high enhancement potential in future with lesser efforts.

FUTURE SCOPE OF THE WORK

The Future Scopes (or proposed Future Works) of this Project for enhanced performance include:

- Integrating IVAN AI Pipeline with the GovAI-Sub Web App for making the full system AI enabled for achieving the actual Automated Form Filling experience by leveraging proper GPU enabled Environments.
- Enabling Multilingual Interface for improved accessibility with multilingual Form support and integrate a Multilingual Virtual Keyboard feature into the GovAI-Sub Web App for easier typing in Indic Languages, enhancing the form-filling process for diverse group of users.
- Enabling the direct submission of forms to Government portals and enabling the Virtual Buffer feature in the GovAI-Sub web app for improved user experience.
- Incorporating proper Handwritten support in OCR for efficient data extraction from Multilingual Handwritten Documents.
- Data Augmentation with noisy OCR-samples to expand IVAN-NER Dataset to approximately 1000 samples covering more diverse document types and entities, with legal permissions for the same to replicate government document data into a dataset.
- Replace outdated PolyGlott module with a modern SOTA language detection module.
- Mitigate translation and ASR errors to transcriptions, noise, accent, numeric-strings, etc., with simple spell-checkers and rule based approaches for simplicity or advanced AIML based approaches.
- Improve OCR performance on degraded documents with explicit image preprocessing steps as mentioned in Super Resolution [7] or PreP-OCR [11], that are absent in current IVAN-OCR framework.
- Use better evaluation strategy for OCR testing focusing solely on Information Retrieval principles, on the degree of necessary information retained by OCR, rather than penalizing the module for basic document texts in output that do not cater to important information for form-filling.
- Focus on designing security, privacy and edge deployment principles and strategies for IVAN AI, accounting for real-world usage in future.

REFERENCES

- [1] "India Population (2025)," Worldometers, 2024. [Online]. Available: <https://www.worldometers.info/world-population/india-population/> [Accessed: Dec. 22, 2025]
- [2] "2011 Census of India," Office of the Registrar General & Census Commissioner, Government of India, 2011. [Online]. Available: <http://censusindia.gov.in> [Accessed: Dec. 22, 2025]
- [3] "Digital India Annual Report 2024-2025," Ministry of Electronics and Information Technology, Government of India, 2025. [Online]. Available: <https://www.meity.gov.in>
- [4] "Rural Youth Lead India's Digital Transformation," Press Information Bureau, Government of India, Oct. 21, 2024. [Online]. Available: <https://www.pib.gov.in/PressNoteDetails.aspx?NoteId=153358> [Accessed: Dec. 22, 2025]
- [5] "User Experience for Government Websites & Apps (UX4G)," National e-Governance Division, Ministry of Electronics and Information Technology, Government of India, 2025. [Online]. Available: <https://www.ux4g.gov.in> [Accessed: Dec. 22, 2025].
- [6] A. R. Hegde et al., "Automated government form filling for aged and monolingual people using interactive tool," Disability and Rehabilitation: Assistive Technology, vol. 19, no. 6, pp. 2260-2270, 2024. [Online]. Available: <https://doi.org/10.1080/17483107.2023.2277825>
- [7] Lat, A., & Jawahar, C. V. "Enhancing OCR Accuracy with Super Resolution," 2018 24th International Conference on Pattern Recognition (ICPR), pp. 3162-3167, 2018. IEEE. [Online]. Available: <https://ieeexplore.ieee.org/document/8545609>
- [8] "OCR Generated Text Summarization using BART," ResearchGate, 2024. [Online]. Available: https://www.researchgate.net/publication/392564275_OCR_Generated_Text_Summ arization_using_BART
- [9] "AI-Based OCR Solution for Aadhaar Verification," RPA CPC, 2024. [Online]. Available: <https://rpacpc.com/ai-based-ocr-solution-for-aadhaar-verification/>

- [10] Dong, Y., Ueda, N., Boros, K., Ito, D., Sera, T., & Oyamada, M. "SCAN: Semantic Document Layout Analysis for Textual and Visual Retrieval-Augmented Generation," arXiv, vol. abs/2505.14381, pp. 1-20, May 2025. [Online]. Available: <https://arxiv.org/html/2505.14381v2>
- [11] Guan, S. et al., "PreP-OCR: A Complete Pipeline for Document Image Restoration and Enhanced OCR Accuracy," arXiv, vol. abs/2505.20429, pp. 1-17, May 2025. [Online]. Available: <https://arxiv.org/abs/2505.20429>
- [12] R. Verma, V. Sharma, M. Ravikiran, R. Saluja, and L. Behera, "Literally reading behind the lines: A benchmark for OCR on cluttered printed documents", unpublished.
- [13] Tewani, R., Dubey, A. K., Jain, A., Agarwal, E., and Mittal, D., "An efficient extraction of information from Indian Government issued documents Aadhar and Pan Card," Fusion Practice and Applications, vol. 4, no. 2, pp. 56-61, 2021. [Online]. Available: <https://doi.org/10.54216/FPA.040201>
- [14] Raghuvanshi, Prof., Girase, S., Patil, S., Chaudhari, S., and Ahirrao, M., "An End-to-End AI System for Automated Verification of Indian Identity Documents Using OCR and Large Language Models," International Journal of Advanced Research in Science, Communication and Technology, vol. 5, pp. 160-169, 2025. [Online]. Available: <https://doi.org/10.48175/IJARSCT-29919>
- [15] Narendranaath, S. R., Muralidharan, S., Krishna Sai Ram, R., and Prema, V., "Automated OCR-Based PAN Card Text Extraction System," International Journal for Research Trends and Innovation (IJRTI), vol. 10, no. 3, pp. b47-b53, Mar. 2025. ISSN: 2456-3315. [Online]. Available: <https://www.ijrti.org/papers/IJRTI2503107.pdf>
- [16] R. K. Komma, "AI-Powered Vendor Form Automation: A Comprehensive Technical Analysis of Enterprise Implementation," International Journal of Scientific Research in Computer Science, Engineering and Information Technology, vol. 11, no. 1, pp. 2975-2983, Jan.-Feb. 2025. [Online]. Available: <https://doi.org/10.32628/CSEIT251112285>
- [17] Z. Zhang, W. You, T. Wu, X. Wang, J. Li, and M. Zhang, "A Survey of Generative Information Extraction," in Proceedings of the 31st International Conference on Computational Linguistics, Jan. 2025, pp. 4840-4870. [Online]. Available: <https://aclanthology.org/2025.coling-main.324.pdf>

- [18] Yanamaddi, L. G., & Kummari, B. "Vision-guided automation: A generic approach to web form filling using GPT and computer vision," World Journal of Advanced Research and Reviews, vol. 20, no. 03, pp. 2096-2107, 2023. [Online]. Available: <https://doi.org/10.30574/wjarr.2023.20.3.2524>
- [19] Ghosh, A., Acharya, A., Saha, S., & Jain, V. "Exploring the Frontier of Vision-Language Models: A Survey of Current Methodologies and Future Directions," arXiv, vol. abs/2404.07214, pp. 1-32, Apr. 2024. [Online]. Available: <https://arxiv.org/html/2404.07214v2>
- [20] Jeong, H. S., Jo, S., Yoon, B. H., Heo, Y., Jeong, H., & Kim, T. "Zero-Shot Document Understanding using Pseudo Table of Contents-Guided Retrieval-Augmented Generation," arXiv, vol. abs/2507.23217, pp. 1-20, Jul. 2025. [Online]. Available: <https://www.arxiv.org/pdf/2507.23217>
- [21] P. Krishnamurthy and R. Srirama, "Form-Filling on Autopilot: How Generative AI and Amazon Connect are Transforming Agent Workflows," International Journal of Emerging Trends in Computer Science and Information Technology, vol. 5, no. 2, pp. 103-112, 2024. [Online]. Available: <https://doi.org/10.63282/3050-9246.IJETCSIT-V5I2P111>
- [22] Rastogi, A. K., "FormFillBharat: Building an AI-Powered Government Form Autofiller Using AWS Textract, Amazon Bedrock & Kiro (AI for Bharat Challenge)," AWS Builder Center, Nov. 2025. [Online]. Available: <https://builder.aws.com/content/36CqsApTafHfrGyf8vjbO4zgtyj/formfill-bharat-building-an-ai-powered-government-form-autofiller-using-aws-textract-amazon-bedrock-and-kiro-ai-for-bharat-challenge>
- [23] Reddy, A. "I Built DocuMind: A Local AI Tool for PAN Card and Aadhaar Card Data Extraction," Medium, Oct. 2025. [Online]. Available: <https://share.google/mQ9EPA2BGlh4jBHzw>
- [24] "AI Autofill for the Web: Smarter Automation for Filling Forms," MVST Blog, 2024. [Online]. Available: <https://www.mvst.co/blog/post/ai-autofill-for-the-web-smarter-automation-for-filling-forms>
- [25] "Automate Form Filling with Skyvern: AI Browser Automation," Skyvern Blog, 2024. [Online]. Available: <https://www.skyvern.com/blog/automate-form-filling-with-skyvern-ai-browser-automation/>
- [26] B. Li, et al., "FormFactory: An Interactive Benchmarking Suite for Multimodal Form-Filling Agents," arXiv, vol. abs/2506.01520, pp. 1-12, Jun. 2025. [Online]. Available: <https://arxiv.org/abs/2506.01520>

- [27] K. S. Bhogale, S. Sundaresan, A. Raman, T. Javed, M. M. Khapra, and P. Kumar, "Vistaar: Diverse Benchmarks and Training Sets for Indian Language ASR," arXiv, vol. abs/2305.15386, pp. 1-6, Aug. 2023. [Online]. Available: <https://arxiv.org/abs/2305.15386>
- [28] Belgacem, H., Li, X., Bianculli, D., & Briand, L. C. "A Machine Learning Approach for Automated Filling of Categorical Fields in Data Entry Forms," ACM Trans. Softw. Eng. Methodol., vol. 1, no. 1, Article 1, 2022, 40 pages. [Online]. Available: <https://arxiv.org/abs/2202.08572>
- [29] W. Syed, "AI-Powered Multi-Modal Form Filling: Advancing Accessibility through Voice and Image Recognition," International Journal of Scientific Research in Computer Science, Engineering and Information Technology, vol. 11, no. 1, pp. 01-11, Jan.-Feb. 2025. [Online]. Available: <https://doi.org/10.32628/CSEIT25111203>
- [30] Sebastian, G. P., & Musthafa, T. "Automated Form Filling Using OCR and Machine Learning for Enhanced Data Accuracy and Efficiency," International Journal of Research in Academic World, vol. 3, no. 6, pp. 67-75, 2024. [Online]. Available: <https://academicjournal.ijraw.com/>
- [31] Khurana, D., Koli, A., Khatter, K., & Singh, S. "Natural language processing: State of the art, current trends and challenges," Multimedia Tools and Applications, vol. 82, no. 3, pp. 3713-3744, 2023. [Online]. Available: <https://link.springer.com/article/10.1007/s11042-022-13428-4>
- [32] "Large Language Models in Document Intelligence," ACM Digital Library, 2024. [Online]. Available: <https://dl.acm.org/doi/abs/10.1145/3768156>
- [33] "Automated Information Extraction System," DataToBiz, 2024. [Online]. Available: <https://www.datatobiz.com/blog/automated-information-extraction-system/>
- [34] Fu, L. et al., "OCRBench v2: An Improved Benchmark for Evaluating Large Multimodal Models on Visual Text Localization and Reasoning," arXiv, vol. abs/2501.00321, pp. 1-26, Jan. 2025. [Online]. Available: <https://arxiv.org/html/2501.00321v2>
- [35] Paruchuri, V. and Datalab Team, "Surya: OCR, Layout Analysis, Reading Order, Table Recognition in 90+ Languages," GitHub repository, 2025. [Online]. Available: <https://github.com/VikParuchuri/surya>
- [36] PaddlePaddle, "PaddleOCR Documentation - Official Website," 2025. [Online]. Available: <http://www.paddleocr.ai/main/en/index.html>

- [37] PolyGlot NLP Library, Documentation, 2021. Available at: <https://PolyGlot.readthedocs.io/en/latest/>
- [38] Meta AI Research, "No Language Left Behind (NLLB)," GitHub repository, 2022. [Online]. Available: <https://github.com/facebookresearch/fairseq/tree/nllb>
- [39] Radford, A., Kim, J. W., Xu, T., Brockman, G., McLeavey, C., and Sutskever, I., "Whisper: Robust Speech Recognition via Large-Scale Weak Supervision," GitHub repository, 2022. [Online]. Available: <https://github.com/openai/whisper>
- [40] Hugging Face, "BERT official documentation (transformers v4.21.2)," Hugging Face Docs, 2024. [Online]. Available: https://huggingface.co/docs/transformers/v4.21.2/en/model_doc/ber
- [41] dsllim, "bert-large-NER: Fine-tuned BERT Model for Named Entity Recognition," Hugging Face Model Hub, 2021. [Online]. Available: <https://huggingface.co/dsllim/bert-large-NER>
- [42] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., and Artzi, Y., "BERTScore: Evaluating Text Generation with BERT," in Proc. Int. Conf. Learn. Represent. (ICLR), 2020. [Online]. Available: <https://arxiv.org/abs/1904.09675>
- [43] Django Software Foundation, "Cross Site Request Forgery (CSRF) Protection – Django Documentation," Version 5.2, Available at: <https://docs.djangoproject.com/en/5.2/howto/csrf/>
- [44] Mozilla Developer Network (MDN), "HTML Documentation," Available at: <https://developer.mozilla.org/en-US/docs/Web/HTML>
- [45] Mozilla Developer Network (MDN), "CSS Documentation," Available at: <https://developer.mozilla.org/en-US/docs/Web/CSS>
- [46] Mozilla Developer Network (MDN), "JavaScript Documentation," Available at: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>
- [47] Parallax, "jsPDF Library," Available at: <https://parall.ax/products/jspdf>

APPENDIX

- The GovAI-Sub Web Interface:

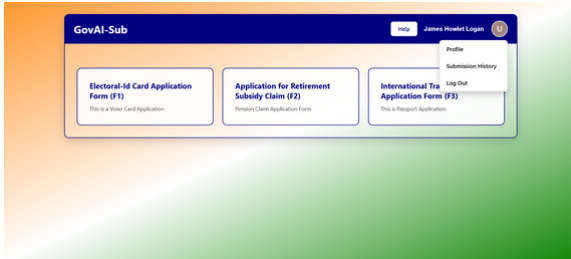


Fig 1: Public Home Page
(Home Page)

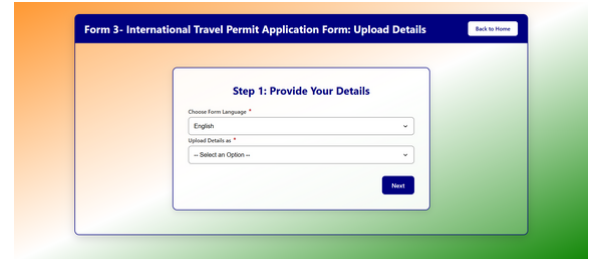


Fig 2: Language & Modality Page
(currently inactive)

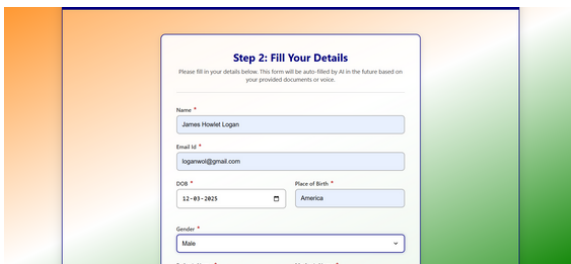


Fig 3: Review, Edit and Submit Page
(the auto-filling happens here)

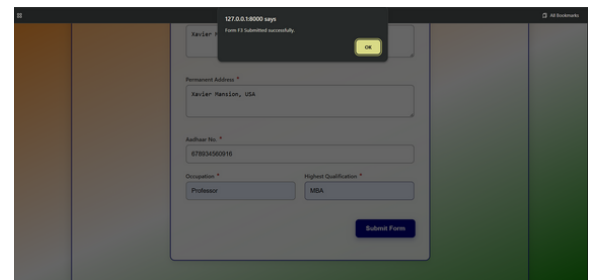


Fig 4: Easy Form Submissions
(Saved in Database, Portal Submission inactive)



Fig 5: Profile Page
(to check saved user data)

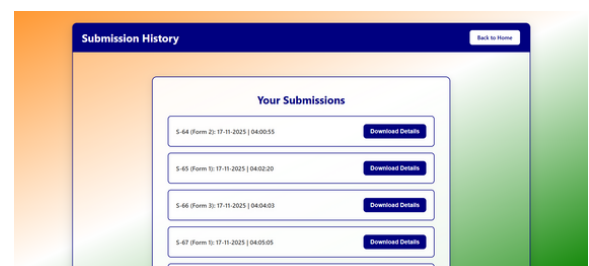


Fig 6: Submission History Page
(for Submission Tracking and Download Details option to download filled Form for further manual processing)

NOTE: ALL THE FORMS SHOWN HERE ARE SUPERFICIAL REPLICAS. THEY DO NOT COPY ANY ACTUAL GOVERNMENT FORMS OR PROPERTY AND ANY SUCH RELEVANCE IS HIGHLY COINCIDENTAL.

- **IVAN-NER Dataset:** 100 text samples derived and replicated from 9 different government documents, annotated using Label Studio in BIO tagging scheme of NER Datasets with 21 form-specific entities, and downloaded in the CoNLL format.

```

|DOCSTART- -X- O
Registration -X- _ O
No. -X- _ O
: -X- _ O
WB.BA.2013.40279 -X- _ O
Case -X- _ O
No. -X- _ O
: -X- _ O
9581 -X- _ O
Memo -X- _ O
No. -X- _ O
164 -X- _ O
Dated -X- _ O
12 -X- _ O
/ -X- _ O
10 -X- _ O
/ -X- _ O
2023 -X- _ O
Date -X- _ O
of -X- _ O
Registration -X- _ O
: -X- _ O
05 -X- _ O
/ -X- _ O
02 -X- _ O
/ -X- _ O
1956 -X- _ O
Name -X- _ O
of -X- _ O
the -X- _ O
Child -X- _ O
: -X- _ O

```

Fig 7: IVAN-NER Dataset